ColdChain Guardian

Virtual IoT cold-storage monitor and cooling controller

Project summary · Firmware v1.0.0-sim · Wokwi ESP32 simulation

DISCLAIMER: This is a software simulation in Wokwi. It is not a physically tested refrigeration unit, not a medical device, and must not be used to control real vaccines, food, or hardware.

1. What this project is

ColdChain Guardian is a virtual Internet-of-Things (IoT) node that watches a cold room and drives a simulated compressor. An ESP32 microcontroller reads a DHT22 temperature/humidity sensor, shows live status on an OLED screen, classifies the temperature as normal / warning / critical, sounds a buzzer, lights status LEDs, and switches a relay that stands in for cooling.

The hardware does not exist as a purchased kit. Everything runs in Wokwi, a free browser simulator: the ESP32, the sensor, the display, the buzzer, the LEDs, and the relay are all virtual parts wired in diagram.json. When you drag the DHT22 temperature slider, the firmware reads that value over the simulated data pin. There is no Python script inventing numbers and no fake dashboard.

There is no phone app and no desktop window besides the Wokwi page (circuit on the right, code on the left, Serial monitor at the bottom).

2. What you received

- ESP32 firmware in C++ (Arduino), split into sketch.ino plus .h/.cpp modules.
- Circuit definition: diagram.json (ESP32 DevKit C v4, DHT22, SSD1306 OLED, LEDs, buzzer, relay, ACK button, fault switch).
- libraries.txt listing Wokwi Arduino libraries.
- Host unit tests for zone classification, latching, recovery, and cooling hysteresis.
- This summary PDF and README.md.

3. How to run it (Wokwi)

You need a browser and an internet connection. No physical board. No paid Wokwi plan is required (free builds may wait in a queue).

1. Open https://wokwi.com and choose New project → ESP32 (Arduino).
2. Sign in if you want to Save and get a shareable project URL.
3. Replace diagram.json, sketch.ino, and libraries.txt with the copies from this folder.
4. Add File for every remaining .h and .cpp in the folder (same names). Arduino/Wokwi compile all of them together, like the Arduino IDE.
5. Confirm Library Manager has: DHT sensor library for ESPx; Adafruit SSD1306; Adafruit GFX Library; Adafruit BusIO; PubSubClient.
6. Press the green Play button. Wait for compile (can take a minute, plus a queue if you are not on a paid plan).
7. Success looks like: OLED “CCG SIM NORMAL”, green LED on, Serial “loop start” and STATUS zone=NORMAL T=5.00.

8. Click the DHT22 body (not a Python file). Use the temperature slider. Keyboard A presses ACK. Fault switch LEFT injects sensor failure; leave it RIGHT for a healthy sensor.

If a .cpp file does not compile, it is not next to sketch.ino — put all firmware files in the same Wokwi project. DHT22 data must be on GPIO 15 (Wokwi's ESP32 DHT examples). GPIO 4 does not decode the sensor in this simulator.

4. Simulated hardware and pins

Part	Wokwi type	ESP32	Role
ESP32 DevKit C v4	board-esp32-devkit-c-v4	—	Microcontroller
DHT22	wokwi-dht22	GPIO 15	Temperature / humidity
SSD1306 OLED 128×64	board-ssd1306	SDA 21, SCL 22	Live UI (I2C 0x3C)
Green / yellow / red LEDs	wokwi-led + 220 Ω	25 / 26 / 27	OK / warning / critical
Piezo buzzer	wokwi-buzzer	GPIO 14	Audible alarm
Relay (PNP, active-high)	wokwi-relay-module	GPIO 13 IN	Simulated compressor
Blue LED on relay NO	LED + 220 Ω from 5 V	via relay	Cooling visible
ACK button (key A)	wokwi-pushbutton	GPIO 32	Mute buzzer
Fault switch	wokwi-slide-switch	GPIO 33	Inject sensor failure

Relay: transistor = pnp so GPIO 13 HIGH means cooling ON (COM connected to NO, blue LED lights). Fault switch RIGHT = healthy (GPIO 33 pulled up). LEFT shorts GPIO 33 to GND = injected failure.

5. Firmware behaviour

The loop in sketch.ino never waits for Wi-Fi. Cooling and alarms keep running if MQTT is down.

1. Read DHT22 about once per second (DHTesp). Invalid CRC/NaN increments a fail streak.
2. Three failed reads, or the fault switch left, → zone FAULT. Cooling stays ON (fail-safe). Alarm latches.
3. Valid temperature: WARNING at ≤ 2 °C or ≥ 8 °C; CRITICAL at ≤ 0 °C or ≥ 10 °C. Leaving a worse zone needs 0.5 °C hysteresis.
4. Excursion timer starts when the zone leaves NORMAL (or on fault). OLED shows EXC mm:ss.
5. Persistent alarm: warning held ≥ 15 s, or critical held ≥ 3 s, or any sensor fault.
6. ACK mutes the buzzer only. Latch and LEDs remain until recovery.
7. Auto-recovery: temperature must stay in-band (after hysteresis) for 4 s. Then latch, timer, and mute clear. Green LED blinks during the hold.
8. Cooling relay is separate hysteresis: ON at ≥ 6 °C, OFF at ≤ 4 °C. Default sensor is 5 °C, so cooling starts OFF.

6. Default thresholds

Parameter	Default	Serial set key
Warning low / high	2 / 8 °C	warn_low warn_high
Critical low / high	0 / 10 °C	crit_low crit_high
Zone hysteresis	0.5 °C	hyst

Cool ON / OFF	6 / 4 °C	cool_on cool_off
Warning latch	15000 ms	warn_latch_ms
Critical latch	3000 ms	crit_latch_ms
Recovery hold	4000 ms	recovery_ms

Serial is 115200 baud. Commands: help, status, ack, reset, set warn_high 9.

7. Demo scenarios (what to show)

Start from ~5 °C, fault switch RIGHT, after auto-recovered or Serial reset.

Scenario	What you do	What you should see
Normal	Leave ~5 °C	Green LED, OLED NORMAL, COOL OFF, quiet buzzer
Warning	Slide to ~8.5 °C	Yellow LED, WARNING, timer counts, cooling ON
Prolonged	Hold 8.5 °C > 15 s, then A / ACK	ALM LATCH, then mute with latch still on
Critical	Slide to ~11 °C (or below 0 °C)	Red LED, CRITICAL, latch in ~3 s, cooling ON
Sensor fail	Fault switch LEFT	FAULT, T ---, red blink, cooling ON
Recovery	Switch RIGHT, slide to ~5 °C, wait >	Green blink RCV, then NORMAL, latch cleared

Hysteresis note: leaving warning at 8.5 °C requires dropping to ≤ 7.5 °C. Cooling stays ON until ≤ 4 °C; at 5 °C after a hot excursion it may still be ON until you dip to ~3.5 °C.

8. Optional Wi-Fi / MQTT

This is extra. The assignment still passes on OLED/LEDs/relay if MQTT fails. Wokwi open AP: SSID Wokwi-GUEST, no password, channel 6.

- Broker: broker.hivemq.com port 1883
- Publish: coldchain/guardian/telemetry (JSON, sim:true)
- Subscribe: coldchain/guardian/cmd (ack, reset, set warn_high 9)

On a laptop you can use the HiveMQ public websocket client, connect, and subscribe to the telemetry topic. To compile without MQTT, set CCG_ENABLE_MQTT to 0 in config.h.

9. Source files

File	Role
sketch.ino	setup() / loop() glue
config.h	Pins and default thresholds
types.h	Zone enum and system state
sensor_reader.*	DHT22 + fail injection
zone_logic.*	Hysteresis, latch, recovery
cooling.*	Relay setpoints
actuators.*	LEDs, buzzer, relay pin
ui_display.*	OLED
inputs.*	ACK + fault switch

serial_cli.*	Runtime config over Serial
telemetry.*	Optional Wi-Fi / MQTT
diagram.json	Circuit
libraries.txt	Wokwi libraries
tests/test_zone.cpp	Host tests (no ESP32)

Host tests (no board):

```
g++ -std=c++17 -I. -Itests/arduino_stub tests/test_zone.cpp zone_logic.cpp cooling.cpp -o /tmp/test_zone && /tmp/test_zone
```

10. What was verified

- Host tests: normal, warning, hysteresis, 15 s latch, ACK mute, critical, 4 s recovery, sensor-fault failsafe cooling, relay 6/4 °C. All passed.
- Wokwi: firmware compiled and ran. At 5 °C: NORMAL, green LED, OLED live, cooling off, Wi-Fi Wokwi-GUEST and MQTT connected.
- DHT22 on GPIO 4 did not decode in Wokwi; GPIO 15 does. That is the shipping pin map.

11. What this is not

- Not a tested physical fridge or vaccine refrigerator.
- Not a mobile app. Not a cloud product. MQTT is a demo on a public broker.
- No hardware was purchased. Do not load this firmware onto a real compressor controller.

Course / demo simulation only. No warranty.